

Computer Hydrological Forecasting

Forecasting river flow from its own past discharge (a statistical approach)

Author: Ugbodaga Benedict Osikpemi (190402003)

Department: Civil and Environmental Engineering

Institution: University of Lagos

Supervisor: Prof. K. O. Aiyesimoju

Submission: February 2026

This document explains the project in three parts: first in very simple terms, then the real technical version, and finally how to actually use the software.

0. What This Version Changed (and Why)

An earlier version of this project used a rainfall-runoff model (a two-bucket conceptual model driven by rainfall and temperature, with parameters transferred to an ungauged Nigerian basin). Following the supervisor's guidance, the project was revised to a stronger, more statistical footing:

- **Discharge only:** Rainfall, temperature and evapotranspiration are no longer used. The model forecasts discharge from past discharge alone.
- **No rainfall-runoff:** The unit-hydrograph / rainfall-runoff transform was removed and replaced by a statistical time-series model (ARIMA).
- **One basin:** The Nigerian (Ogun-Osun) catchment was dropped. The Conecuh River (Alabama, USA) is now the single study basin.
- **More rigorous:** The whole workflow is defined statistically: stationarity tests, autocorrelation-based identification, information-criterion model selection, and residual diagnostics.

In short: instead of turning rainfall into runoff, the model now learns the river's own day-to-day behaviour from its history and projects it forward - a cleaner, more defensible, and reproducible approach.

1. The Simple Version (for anyone)

A river does not change its flow randomly from one day to the next. If a lot of water is flowing today, a lot will still be flowing tomorrow; if the river is low today, it will most likely be low tomorrow. Flow changes gradually, following a recession after each rise.

This project is a computer program that uses that simple fact. It looks at the recent history of the river's flow and answers one question: "Given how the river has been flowing, how much water will flow over the next few days?" It needs no weather information at all - only the river's own past.

Learning the river's habits

The program studies many years of daily flow measurements and learns the pattern: how strongly today's flow depends on yesterday's, the day before, and so on. It captures that pattern in a small set of numbers and then uses them to step forward into the future, one day at a time.

How do we know it works?

We test it honestly. There is a very simple guess called "persistence": assume tomorrow's flow equals today's. Because rivers change slowly, that guess is already quite good - so any real model must beat it. Our model does beat persistence at every forecast range we tested, which proves it is adding genuine skill and not just repeating the obvious.

2. The Actual Version (technical)

The project is a modular, open-source Python framework for short-term streamflow forecasting (1-3 days ahead) from a univariate daily discharge series. It uses the ARIMA (autoregressive integrated moving average) class of models formalised by Box and Jenkins (1976). No exogenous (meteorological) inputs are used.

2.1 The data

Daily mean discharge for the Conecuh River, Alabama (USGS gauge 02361000) from the CAMELS US data set, 1980-2014 (12,784 days). Discharge is converted from cubic feet per second to cubic metres per second, the few missing values (<1%) are interpolated, and the series is modelled on the natural-log scale to stabilise its variance. The record is split into a training period (1980-2003) and an independent validation period (2004-2014).

2.2 The model: ARIMA(p, d, q)

An ARIMA model describes a series using three ingredients: an autoregressive part (the value depends on its own p past values), an integration order d (the series is differenced d times to make it stationary), and a moving-average part (the value depends on the q past random shocks). On the differenced log-series $w(t)$:

$$w(t) = c + \phi_1 w(t-1) + \dots + \phi_p w(t-p) + a(t) + \theta_1 a(t-1) + \dots + \theta_q a(t-q)$$

where ϕ are the autoregressive coefficients, θ the moving-average coefficients, c a constant, and $a(t)$ a white-noise error term.

2.3 How the model was identified (Box-Jenkins)

- **Stationarity:** Augmented Dickey-Fuller and KPSS tests determined the differencing order: one difference ($d = 1$) makes the series stationary.
- **Identification:** The autocorrelation (ACF) and partial autocorrelation (PACF) functions suggested candidate AR and MA orders.
- **Estimation:** Coefficients were estimated by conditional sum of squares (pure AR models by exact least squares).
- **Selection:** All candidate orders were ranked by the Akaike Information Criterion (AIC). The winner was ARIMA(3, 1, 2).
- **Diagnostics:** The Ljung-Box test confirmed the residuals are white noise ($p = 0.23$), so the model is statistically adequate.

2.4 Results

Forecasts were evaluated by rolling-origin (walk-forward) testing over the validation period and benchmarked against persistence. The model beats persistence at every lead time (positive

persistence skill score, PSS):

Lead	Model NSE	PSS (skill vs persistence)	Persistence NSE
1 day	0.824	+0.224	0.773
2 day	0.541	+0.244	0.393
3 day	0.347	+0.267	0.109

Forecasts are back-transformed from the log scale with a log-normal bias correction, so the percentage bias is near zero at all lead times. The one-day forecast is rated "very good" on the Moriasi et al. (2007) scale; absolute skill declines with lead time - expected for any model that uses only past flow - so useful skill is concentrated at about the one-day horizon. The raw NSE is high partly because daily flow is strongly autocorrelated, so the persistence skill score is the decisive, honest measure of the value added.

2.5 Code structure

```
src/preprocess.py - load discharge, unit conversion, log transform, split
src/model.py      - ARIMA + ADF/KPSS/ACF/PACF/Ljung-Box (from scratch)
src/calibrate.py  - stationarity tests + AIC order selection
src/forecast.py   - rolling multi-step forecast + persistence benchmark
src/metrics.py    - NSE, RMSE, PBIAS, MAE, R2, skill score
src/plots.py      - the six figures
run_pipeline.py   - runs everything end to end
write_document.py - builds the thesis .docx
app.py + pages/   - the Streamlit web app
```

The entire time-series toolkit is implemented directly from its defining equations on NumPy and SciPy, so the framework is fully self-contained and reproducible (no external time-series library is required).

3. How to Use the Software

3.0 The easy version

Using the tool is like checking a forecast: you open the app, choose how many days ahead you want, and it shows the predicted river flow. You do not enter any weather - the model works from the river's own record. Here is all you do:

- **1.** Open the app - a page opens in your web browser.
- **2.** On the left, choose how many days ahead you want to forecast (1-7).
- **3.** Choose how much past history to display on the chart.
- **4.** Press the "Run Forecast" button.
- **5.** Read the answer: predicted flow per day as cards, a table, and a chart, plus the model's validation skill.

3.1 The web app, step by step (the main way to use it)

Open the app and you get a browser page with two pages in the left sidebar: "Forecast Tool" and "Documentation". On the Forecast Tool page:

- **1.** Set the forecast Horizon (1 to 7 days).
- **2.** Set how many days of history to display.
- **3.** Click "Run Forecast".
- **4.** Read the results: per-day summary cards, a forecast table (with the record-mean reference), a chart of recent history plus the forecast, and the out-of-sample validation skill (NSE and persistence skill score by lead time).

3.2 The full run (get every chart and number at once)

Run the whole study in one command. It loads the discharge record, runs the stationarity tests, identifies and estimates the ARIMA model, performs the rolling forecast evaluation, and saves all six figures plus a results file (data/results.json). The terminal prints the chosen model, the metric table and the residual diagnostic.

```
python run_pipeline.py      # model + forecasts + figures + results.json
python write_document.py    # rebuild the thesis .docx from those results
streamlit run app.py        # launch the interactive web app
```